Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC2006 – Programación Orientada a Objetos · Semestre II – 2026

Exotic Ores

Informe consolidado del proyecto

Entrega final

David Adriel Aguirre Leonzo (261134)

Allan Enrique Ronneseth Quemé (26493)

Fabio Roberto Calvo Lemus (262256)

Repositorio: https://github.com/astxrfr/ExoticOres.git

Guatemala, noviembre de 2026

1. Introducción, problema inicial y solución planteada

1.1 Descripción del proyecto

Exotic Ores es un mod de Minecraft: Java Edition, desarrollado sobre NeoForge, que amplía la progresión de la dimensión del End. El proyecto parte de un material radiactivo, la Thorite, que aparece de forma natural en el End y que obliga al jugador a protegerse, y alrededor de él se construyó una cadena de contenido: radiación con intensidad según la distancia, pantalla y sonido que avisan del peligro, armaduras que protegen, un segundo material (el plomo) que se obtiene en el Overworld y sirve para fabricar el equipo, y, en las últimas entregas, armas con efecto de veneno, nuevos bloques y una cura para la radiación.

Este informe reúne y enlaza la documentación de todas las entregas del semestre. Entre la segunda y la última entrega el mod pasó de un único bloque radiactivo con su entidad de bloque a un sistema con más de veinte requisitos funcionales. Para facilitar la lectura, el documento sigue el hilo del proyecto: primero el problema inicial y la solución planteada con design thinking, luego la tecnología elegida, después los requisitos y cómo fueron cumpliéndose, la planificación de cada entrega, la implementación y las pruebas, y finalmente la reflexión del equipo, la opinión de los usuarios y la gestión del tiempo.

La Entrega 1 se dedicó al design thinking: se definió el problema, se entrevistó a jugadores, se eligió la idea y se probaron prototipos con usuarios. Con esa base, la Entrega 2 formuló los requisitos funcionales y construyó el primer núcleo del mod, y desde entonces cada entrega agregó una capa nueva sobre lo anterior, de modo que los requisitos de una etapa casi siempre dependen de los de la etapa previa.

1.2 Problema para resolver

Minecraft presenta un problema de progresión en su etapa final, especialmente en la dimensión del End. Aunque representa el objetivo principal del juego base, esta dimensión ofrece poco contenido más allá del combate contra el Dragón del End y la exploración de las ciudades del End. Como resultado, una vez alcanzado este punto, la experiencia se vuelve fácil, lineal y con escasos incentivos para seguir explorando o enfrentando nuevos desafíos.

El problema existe porque el contenido del End no evoluciona al mismo ritmo que el progreso del jugador, lo que reduce la sensación de descubrimiento y reto que caracteriza las primeras etapas del juego. Resolverlo es importante porque permitiría mantener el interés de los jugadores durante más tiempo, ampliar la experiencia de juego y ofrecer una progresión más atractiva y satisfactoria en la etapa final.

Brief

Desarrollar un mod para Minecraft que amplíe y profundice la experiencia en la dimensión del End mediante nuevos desafíos, mecánicas y contenido. El proyecto busca mejorar la progresión del juego en su etapa final, ofreciendo una experiencia más dinámica y menos lineal.

1.3 Empatizar

Se definieron dos perfiles de usuario con necesidades y frustraciones distintas:

	Jugador principiante	Jugador veterano
Edad	12 a 18 años	16 a 25 años
Rol	Jugador de Minecraft	Jugador experimentado de Minecraft
Nivel tecnológico	Básico	Básico a intermedio
Plataforma	Minecraft Java Edition	Minecraft Java Edition
Necesidades	Más contenido que haga interesante la progresión sin complicar demasiado la experiencia.	Nuevos retos, materiales y mecánicas que prolonguen la vida útil del juego.
Frustraciones	Considera que el contenido del End es limitado y que la progresión termina demasiado rápido.	Percibe que, después de obtener la armadura de netherita y derrotar al dragón, quedan pocos objetivos relevantes.

Tabla. Perfiles de usuario.

A ambos perfiles se les hizo una entrevista con el siguiente guion: (1) qué disfrutan más y menos de la progresión de Minecraft; (2) en qué momento el juego deja de ser desafiante o interesante y por qué; (3) qué contenido o mecánicas les gustaría que se agregaran; (4) qué opinan de la idea de un nuevo mineral radiactivo que ofrezca grandes beneficios, pero también riesgos; y (5) qué característica agregarían a ese mineral o a su mecánica.

1.4 Definir

Perfil experimentado. Considera que Minecraft pierde interés una vez que se alcanza el contenido del End, porque la progresión se vuelve corta y poco desafiante. Valora las mecánicas complejas, el contenido con utilidad real y los retos que exigen estrategia y habilidad, por lo que suele recurrir a mods. Opina que los elementos nuevos deben estar bien equilibrados y ser coherentes con las mecánicas existentes.

Perfil novato. Disfruta recolectar recursos y mejorar su equipamiento, pero le frustra perder materiales o repetir tareas como la minería durante mucho tiempo. Le interesan las mecánicas que faciliten la automatización y la gestión de recursos, y los minerales especiales que den beneficios importantes a cambio de asumir riesgos, siempre que aporten sensación de progreso y recompensa.

Con esa información se elaboró un mapa de empatía para cada perfil.

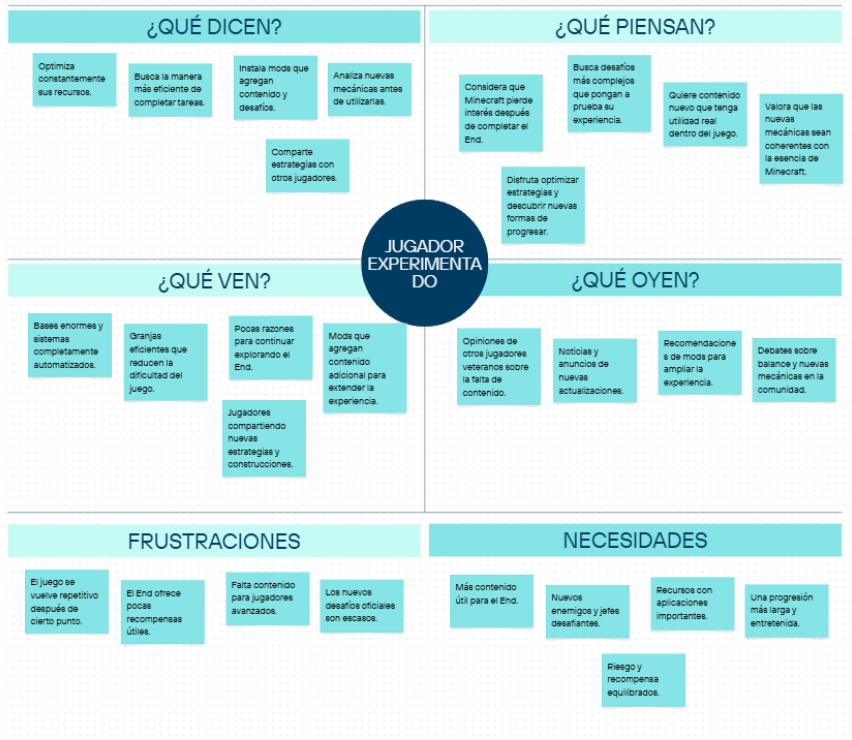


Figura. Mapa de empatía del primer perfil.

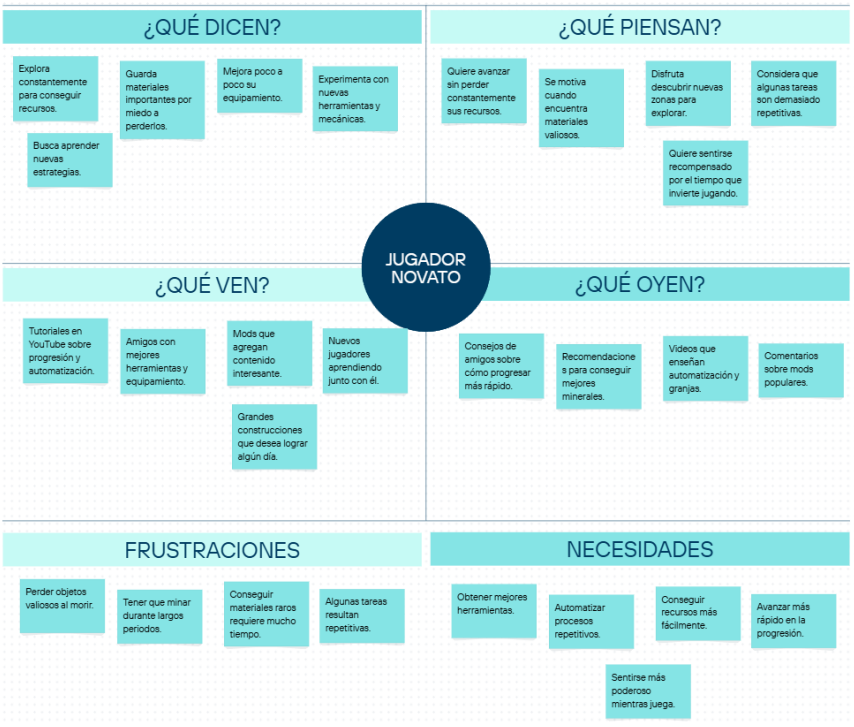


Figura. Mapa de empatía del segundo perfil.

Del análisis se concluyó que tanto jugadores experimentados como novatos consideran que el End posee muy poco contenido y pocos incentivos para seguir explorando después de completar el objetivo principal. Por eso el proyecto consiste en un mod que amplía la experiencia del End con nuevos biomas, recursos, enemigos, estructuras y herramientas, manteniendo el equilibrio

del juego original: nuevos desafíos para los veteranos y una experiencia más atractiva para quienes empiezan.

1.5 Ideación y solución planteada

Se utilizó la lluvia de ideas. Cada integrante propuso funcionalidades que mejoraran el End sin afectar la esencia de Minecraft, se discutieron sus ventajas y desventajas y se votó. Las tres ideas más votadas fueron:

- **Idea 1 (elegida):** expandir el End con nuevos biomas, minerales exclusivos, estructuras, criaturas y herramientas que permitan una progresión más profunda y recompensen la exploración.
- **Idea 2:** crear un sistema de habilidades desbloqueables mediante reliquias obtenidas en el End, con mejoras permanentes para el jugador.
- **Idea 3:** agregar eventos dinámicos en el End, como invasiones, jefes opcionales y mazmorras con recompensas únicas.

Se eligió la primera porque responde directamente a las necesidades de las entrevistas: da retos y contenido útil a los veteranos y más recompensas por explorar y nuevas herramientas a los novatos, y mantiene la estética y la filosofía del juego original. Esa idea es el origen de Exotic Ores y de su mineral radiactivo, la Thorite.

1.6 Prototipos y retroalimentación

Se elaboraron tres prototipos del material y se mostraron a posibles usuarios para recoger sus opiniones.

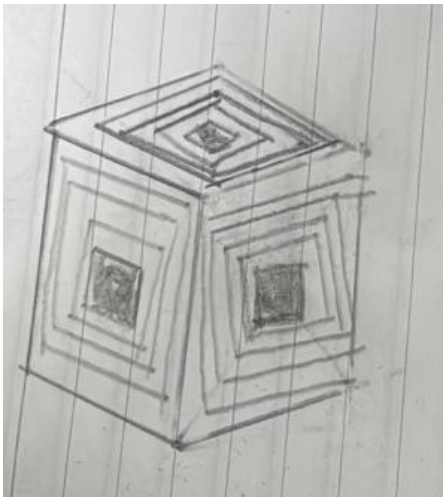


Figura. Prototipo 1.

Les gustó que se acoplaba al tema y parecía parte del End. Criticaron que el diseño era bastante sencillo, preguntaron cuál era el propósito de los cuadrados dentro de cuadrados y sugirieron un diseño mucho más extravagante.

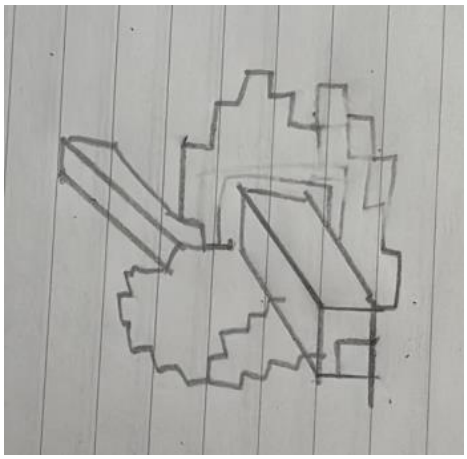


Figura. Prototipo 2.

Les pareció un diseño interesante, pero criticaron que solo por la silueta era poco comprensible y preguntaron por qué parecía tan aleatorio. Pidieron que fuera extravagante pero más comprensible.

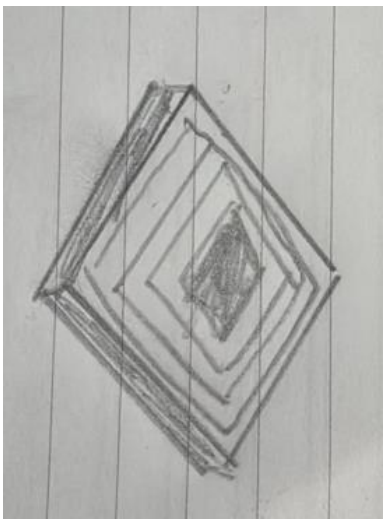


Figura. Prototipo 3.

Les gustó el diseño sencillo pero bonito y no hicieron críticas. Preguntaron cuál era el propósito del material y sugirieron colores vivos tipo neón.

2. Tecnología seleccionada

2.1 Herramientas de desarrollo

- **NeoForge:** framework y mod loader de código abierto para Minecraft: Java Edition, continuación moderna de Minecraft Forge. Se eligió porque ofrece APIs completas para modificar el juego, mantiene compatibilidad con Minecraft 1.21+ y cuenta con un sistema de eventos que facilita implementar efectos como la radiación.
- **Java:** lenguaje de programación del mod, con el que se aplican los conceptos del curso (clases, herencia, interfaces, composición y manejo de excepciones).

- **Blockbench:** aplicación gratuita de modelado 3D de baja poligonización, pensada para Minecraft. Se usa para diseñar modelos y texturas de bloques, ítems y armaduras.
- **Gradle:** herramienta de automatización de construcción y gestión de dependencias; compila el código, administra NeoForge y empaqueta el mod en un JAR.
- **IntelliJ IDEA:** entorno de desarrollo principal, con integración nativa con Gradle, depuración y refactorización.
- **GitHub:** control de versiones. Los tres integrantes trabajan sobre el mismo repositorio.

2.2 Persistencia de datos

El mod usa el sistema de persistencia nativo de Minecraft: NBT (Named Binary Tag) y los archivos de región en formato Anvil. NBT es el formato binario con el que el juego serializa el estado de los bloques con entidad, de las entidades, de los ítems y de los datos del mundo; los archivos de región agrupan los chunks del terreno junto con esos datos. Un BlockEntity ya trae los métodos saveAdditional() y loadAdditional() para guardar su estado en NBT, por lo que clases como RadioactiveBlockEntity no necesitan nada adicional.

Se descartó una base de datos externa por cuatro razones. Primero, es la tecnología que Minecraft y NeoForge exponen para los mods, y reemplazarla implicaría reinventar un mecanismo ya sincronizado con el guardado del mundo. Segundo, todo lo que el mod necesita persistir (el bloque radiactivo, el material recolectado, la armadura equipada, el estado de la mesa de refinado) es por definición estado del mundo del juego. Tercero, NBT/Anvil está optimizado para lecturas y escrituras frecuentes de chunks pequeños, justo el patrón de un bloque que hace tick constantemente; una base de datos externa añadiría dependencias, costo de entrada y salida y problemas de sincronización con el hilo del servidor. Cuarto, el proyecto no maneja datos de usuario que deban consultarse fuera de la partida. Una base de datos aparte habría sido sobreingeniería.

2.3 Patrones y organización del código

El sistema se organizó con el patrón Modelo-Vista-Controlador. El modelo contiene las clases que representan el comportamiento radiactivo (RadiationEffect, RadioactiveBlock, RadioactiveBlockEntity y RadioactiveItem); la vista contiene lo que el jugador ve (ModCreativeTabs y ClientModEvents, y más adelante la capa de pantalla de radiación); y el controlador coordina el registro de contenido (ExoticOres, ModBlocks, ModItems, ModEffects y ModBlockEntities). Además se emplean los registros diferidos de NeoForge (DeferredRegister) para declarar bloques, ítems y efectos, la generación de datos (datagen) para recetas, loot tables, modelos y sonidos, y un sistema de eventos para aplicar o bloquear efectos.

En la quinta entrega se reorganizó la radiación con interfaces (Radioactive, RadiationBlockEmitter) y la clase RadiationExposure, que combina las contribuciones de todas las fuentes y aplica el efecto una sola vez por tick. Con ello cada fuente solo informa su intensidad y la decisión de aplicar daño queda en un único lugar.

2.4 Referencias

Blockbench: modeling, texturing & animating. (s. f.). Bedrock Wiki. https://wiki.bedrock.dev/guide/blockbench

Karkori, M. (s. f.). Gradle para Principiantes 2025: Tu Guía Definitiva para Java y Spring Boot. Mostapha.dev Blog. https://www.mostapha.dev/blog/gradle-principiantes-2025-guia-instalacion-uso-java-spring-boot

The NeoForged Project: Installer and modder information. (2026, 11 de mayo). https://neoforged.net/

The End (biome) – Minecraft Wiki. (s. f.). https://minecraft.wiki/w/The_End_(biome)

Wikipedia contributors. (2026, 8 de agosto). IntelliJ IDEA. Wikipedia. https://en.wikipedia.org/wiki/IntelliJ_IDEA

3. Requisitos funcionales

3.1 Requisitos identificados al inicio (RF-01 a RF-10)

En la Entrega 2 se identificaron diez requisitos funcionales y se priorizaron según su importancia. Los críticos (RF-01, RF-03, RF-04, RF-08 y RF-10) son los esenciales para que el mod funcione; los altos (RF-02, RF-05 y RF-06) enriquecen la experiencia y pueden refinarse después; los medios (RF-07 y RF-09) son mejoras visuales y de configuración.

ID	Requisito	Descripción
RF-01	Generar bloque radiactivo en el End	Generación automática del bloque radiactivo en islas del End con distribución equilibrada.
RF-02	Aplicar efectos de radiación al jugador	Al acercarse al bloque radiactivo, el jugador recibe daño gradual y efectos de radiación.
RF-03	Permitir minería del bloque radiactivo	El bloque puede minarse con herramientas adecuadas y suelta un objeto recolectable.
RF-04	Almacenamiento del material radiactivo	El material se recolecta y se guarda en el inventario con propiedades visuales distintivas.
RF-05	Sistema de daño por radiación	El bloque causa daño gradual al jugador en su proximidad.
RF-06	Efectos visuales y de partículas	El bloque emite partículas radiactivas visibles y tiene un brillo característico.
RF-07	Sonidos específicos	Sonidos característicos al acercarse y al minar el bloque (contador Geiger).
RF-08	Modelo y textura del bloque	Modelo y textura personalizados, coherentes con la estética del End.
RF-09	Sistema de configuración	Parámetros configurables, como la activación de la radiación y su rango.
RF-10	Compatibilidad con NeoForge	Código compatible con NeoForge y Minecraft 1.21+.

3.2 Requisitos que surgieron durante el desarrollo (RF-11 a RF-23)

Conforme el mod creció aparecieron requisitos nuevos. Algunos nacieron de decisiones de diseño (el sistema de armadura y la mesa de refinado, en la Entrega 3); otros, de completar la experiencia de juego (bloque de Thorite, HUD, protección y plomo, en la Entrega 4); y los últimos, de la retroalimentación y del deseo de dar más profundidad a los materiales (armas, texturas, kyanita y cura, en la Entrega 5).

ID	Requisito	Responsable
RF-11	Sistema de armadura de Thorite (material, cuatro piezas y reparación). La mejora sobre armadura de netherita mediante herrería sigue pendiente.	Allan
RF-12	Mesa de refinado de Thorite: convierte el mineral crudo (raw_thorite) en material refinado (thorite_shard).	David
RF-13	Bloque de Thorite y radiación por intensidad, centralizada en RadiationExposure.	David y Fabio
RF-14	Efecto de radiación en la pantalla del jugador (HUD): grano, viñeta e indicador de dosis en mSv/h.	Fabio
RF-15	Protección contra la radiación: armadura de Thorite y traje de radiación de plomo.	Allan
RF-16	Material de plomo: mena, mena de pizarra profunda, plomo en bruto, lingote, generación en el Overworld y bloque de plomo.	David, Allan y Fabio

ID	Requisito	Responsable
RF-17	Herramientas de plomo: pico, hacha, pala y azada.	Allan y David (propuesto)
RF-18	Reservado. Su descripción original debe recuperarse y aclararse antes de planificar trabajo.	—
RF-19	Minibiomas de Thorite en el End, donde se concentre la generación de la mena.	Fabio y David (propuesto)
RF-20	Armas de plomo con efecto Poison al golpear a una entidad susceptible.	Allan
RF-21	Mejora de texturas del bloque de plomo y de la mena de Thorite.	David
RF-22	Bloque de kyanita que puede obtenerse, colocarse y recuperarse según las reglas del material.	David
RF-23	Cura de la radiación e incorporación de un hongo relacionado con esa mecánica.	David, con apoyo de Fabio

3.3 Recuento por entrega

Entrega	Avance en requisitos funcionales
Entrega 1	No incluyó requisitos funcionales: se definieron el problema, los perfiles, la idea elegida y los prototipos, que dieron origen a los requisitos RF-01 a RF-10 de la Entrega 2.
Entrega 2	Se identificaron RF-01 a RF-10, se priorizaron y se diseñó la arquitectura MVC. Quedaron completos RF-03 y RF-04 y se dejaron las bases de RF-01, RF-02, RF-05 y RF-08.
Entrega 3	Completados RF-01, RF-12 y la forma base de RF-11. Mejorados RF-02 y RF-05 (efecto más consistente según la proximidad). Ampliado RF-08 con texturas de armadura, mesa de refinado, mineral crudo y refinado. RF-03 y RF-04 sin cambios; RF-10 se mantiene al actualizar la versión de NeoForge y Minecraft.
Entrega 4	Completados RF-07, RF-09, RF-13, RF-14, RF-15 y RF-16, además de la ampliación de RF-08. Cierre con 14 de 20 requisitos completos, 1 parcial (RF-11) y 5 pendientes (RF-06, RF-17, RF-18, RF-19 y RF-20). De los 10 requisitos iniciales, 9 estaban implementados (90 %).
Entrega 5	Implementado RF-20 (veneno en armas de plomo). Refactorizados RF-02, RF-05 y RF-13 con interfaces y RadiationExposure; ampliados RF-08 y RF-16 (textura y bloque de plomo). Aparecen RF-21, RF-22 y RF-23 como requisitos nuevos en seguimiento.
Entrega final	Cierre del sistema: corrección de errores, uso de la kyanita en la generación, bloques y cofre adicionales y documentación del código (ver sección 5.3).

3.4 Estado de los requisitos

Estado	Requisitos	Cantidad
Completados hasta la Entrega 5	RF-01, RF-02, RF-03, RF-04, RF-05, RF-07, RF-08, RF-09, RF-10, RF-12, RF-13, RF-14, RF-15, RF-16 y RF-20	15
Parcial	RF-11 (armadura de Thorite base; falta la mejora sobre netherita)	1
En seguimiento (iniciados en la Entrega 5)	RF-21, RF-22 y RF-23	3
Pendientes	RF-06, RF-17, RF-18 y RF-19	4

El requisito RF-06 (partículas y brillo) es el único de los diez requisitos iniciales que no se ha implementado, por lo que mientras siga pendiente no se alcanza el cien por ciento de los requisitos del inicio. Los requisitos pendientes se describen con más detalle en la sección 5.5.

4. Planificación

En cada entrega los requisitos se dividieron en tareas pequeñas, cada una con un responsable, horas estimadas y una fecha probable de terminación. Esta sección resume la planificación de las entregas anteriores y detalla la planificación de la entrega final.

4.1 Planificación de la Entrega 1

La primera entrega se organizó en nueve tareas de design thinking, todas con fecha límite el 24 de julio de 2026.

No.	Tarea	Responsable	Fecha
1	Definir el problema	Fabio	24/07/2026
2	Redactar el brief	Fabio	24/07/2026
3	Definir los perfiles de usuario	Allan	24/07/2026
4	Elaborar el guion de entrevistas	Allan	24/07/2026
5	Realizar las entrevistas	Allan	24/07/2026
6	Elaborar los mapas de empatía, insights, necesidades y oportunidades	Fabio	24/07/2026
7	Describir la ideación y las tres ideas más votadas	David	24/07/2026
8	Elaborar los tres prototipos y recopilar la retroalimentación	David	24/07/2026
9	Integrar el documento final y revisar antes de subirlo a Canvas	David	24/07/2026

4.2 Planificación de la Entrega 2

Las ocho tareas giraron alrededor del primer núcleo del mod. Todas tenían como fecha límite el 12 de agosto de 2026.

#	Tarea	Descripción	Responsable	Horas	Fecha
1	Clase RadioactiveBlock	Clase base del bloque radiactivo.	Fabio	4	12/08/2026
2	Diseño de textura en Blockbench	Modelo 3D y texturas del bloque.	David	4	12/08/2026
3	Integración de texturas	Integrar modelo y texturas en el código del bloque.	David y Fabio	4	12/08/2026
4	Sistema de generación en el End	Generación automática del bloque en islas del End.	Fabio, David y Allan	5	12/08/2026
5	Sistema de daño por radiación	Daño y efectos de radiación a jugadores cercanos.	Fabio	12	12/08/2026
6	Pruebas y debugging	Pruebas funcionales y corrección de errores.	Fabio y Allan	3	12/08/2026
7	Documentación UML	Diagrama UML de clases del proyecto.	Fabio y Allan	4–5	12/08/2026
8	Documentación final	Compilación y revisión de la documentación.	Allan	3	12/08/2026

4.3 Planificación de la Entrega 3

La tercera entrega agregó la armadura de Thorite, la mesa de refinado y las texturas asociadas, además de mejorar el efecto de radiación y documentar la elección de la tecnología de persistencia. Las 23 tareas tenían como fecha probable el 24 de septiembre de 2026.

#	Tarea	Responsable	Horas
1	Configurar ConfiguredFeature del mineral (veta en end_stone)	Fabio	1.5
2	Configurar PlacedFeature (frecuencia y cantidad por chunk)	Fabio	1
3	Vincular el mineral al bioma del End (BiomeModifier con IS_END)	Fabio	1
4	Probar la generación en mundo	Fabio	1
5	Revisar y mejorar RadiationEffect	Fabio	2
6	Ajustar el daño gradual por proximidad	Fabio	1.5
7	Probar el efecto mejorado en bloque e ítem	Fabio	1
8	Definir ModArmorMaterial	Allan	2
9	Registrar los 4 ítems de armadura	Allan	2
10	Crear el tag de reparación (THORITE_REPAIRABLE)	Allan	1
11	Probar equipamiento y defensa	Allan	1.5
12	Diseñar el bloque de la mesa de refinado	David	2
13	Registrar el bloque e ítem de la mesa	David	1
14	Implementar la lógica de refinado (raw_thorite a thorite_shard)	David	3
15	Probar el flujo completo de refinado	David	1.5
16	Textura de la armadura	David	3
17	Textura de la mesa de refinado	David	2
18	Textura del mineral crudo y refinado	David	2
19	Integrar las texturas al código	David	1.5
20	Redactar la investigación de tecnología	Allan	1
21	Redactar requisitos y priorización	Allan	1
22	Armar la planificación y el calendario	Allan	1
23	Compilar el documento final	Allan	1

4.4 Planificación de la Entrega 4

La cuarta entrega fue la más extensa: 30 tareas, todas con fecha probable del 15 de octubre de 2026, organizadas en cuatro bloques de trabajo (núcleo de radiación; HUD, sonido y protección; plomo y configuración; texturas y cierre).

#	Tarea	Responsable	Horas
1	Registrar el bloque de Thorite	David	2
2	Crear la receta y la loot table del bloque	David	2
3	Centralizar la radiación en RadiationExposure	Fabio	5
4	Probar el bloque y el ítem radiactivo	David	2
5	Diseñar el overlay de radiación	Fabio	3

#	Tarea	Responsable	Horas
6	Crear el payload de red (RadiationIntensityPayload)	Fabio	3
7	Registrar la capa del HUD (RadiationScreenOverlay)	Fabio	5
8	Probar el HUD	Fabio	2
9	Registrar el sonido del contador Geiger	David	2
10	Reproducir el sonido según la intensidad (GeigerTickClient)	Fabio	3
11	Definir los materiales de armadura	Allan	3
12	Registrar las piezas del traje de radiación	Allan	3
13	Crear el tag y la lógica de protección	Allan	4
14	Agregar inmunidades y restricción de la leche	Allan	3
15	Probar la protección	Allan	2
16	Crear la configuración del mod (ExoticOresConfig)	Allan	3
17	Probar la configuración	Allan	2
18	Registrar el plomo	David	4
19	Crear loot tables, tags y recetas del plomo	Allan	3
20	Generar el plomo en el Overworld	Fabio	4
21	Probar la generación y la obtención	Fabio	2
22	Texturas del plomo	David	4
23	Texturas del traje de radiación	David	5
24	Texturas del HUD	David	3
25	Integrar los modelos y las texturas	David	2
26	Preparar la demostración para usuarios	Fabio	2
27	Grabar el video con los usuarios	David	3
28	Subir el código a GitHub	Allan	2
29	Redactar el documento de la entrega	Allan	3
30	Compilar el documento final	Allan	2

4.5 Planificación de la Entrega 5

En la quinta entrega el trabajo se organizó por requisito y cada tarea quedó vinculada a sus pruebas y a sus commits. Se planificaron 20 tareas (T01 a T20) con fechas entre el 26 de octubre y el 6 de noviembre de 2026.

ID	Tarea	Responsable	Horas	Fecha
T01	Definir el veneno de las armas de plomo	Allan	1	02/11/2026
T02	Agregar Poison al golpear con armas de plomo	Allan	2	02/11/2026
T03	Probar las armas (P06)	Fabio	1.5	05/11/2026
T04	Diseñar el framework de radiación con interfaces	Fabio	2	26/10/2026
T05	Adaptar bloques e ítems (Radioactive, RadiationBlockEmitter, RadiationExposure)	Fabio	4	26/10/2026
T06	Corregir la aplicación del efecto (ModEffectEvents, amplificador, contenedores)	Fabio	3	26/10/2026

ID	Tarea	Responsable	Horas	Fecha
T07	Integrar la recuperación (contributeHeal)	Fabio	2	01/11/2026
T08	Revisar la integración de ramas y versión funcional	Fabio	2	06/11/2026
T09	Probar la radiación y los minerales (P01 a P04)	Fabio	3	06/11/2026
T10	Actualizar las texturas del bloque de plomo y de thorite ore	David	3	05/11/2026
T11	Crear el bloque de plomo	David	2	01/11/2026
T12	Crear el bloque de kyanita	David	3	05/11/2026
T13	Actualizar la cura de la radiación	David	3	05/11/2026
T14	Crear el hongo relacionado con la cura	David	3	05/11/2026
T15	Probar bloques y cura (P07, P08 y P09)	David	2	05/11/2026
T16	Probar el equipamiento (P05)	Fabio	1.5	05/11/2026
T17	Preparar la demostración	Fabio	1	05/11/2026
T18	Grabar a los usuarios	David y Fabio	1	05/11/2026
T19	Relacionar evidencias (requisitos, tareas, commits y pruebas)	Allan	2	05/11/2026
T20	Preparar la entrega (PDF, GitHub y video)	Allan	1.5	05/11/2026

4.6 Planificación de la entrega final

La entrega final se concentra en cerrar el sistema: corregir errores, hacer que la generación use la kyanita, completar los bloques y el cofre pendientes y documentar el código. Cada integrante conserva tareas de programación, y el cierre incluye la muestra a usuarios y la compilación de este informe. Las horas y fechas son estimaciones de planificación.

ID	Tarea	Descripción	Resp.	Horas	Fecha
F01	Corregir errores	Revisar y corregir los errores detectados en las pruebas y en la versión integrada del mod.	Fabio	4	13/11/2026
F02	Generación con kyanita	Hacer que la generación del mod utilice la kyanita (extiende RF-22).	Fabio	4	13/11/2026
F03	Documentar el código (Fabio)	Documentar las clases y métodos de su parte del código.	Fabio	3	13/11/2026
F04	Implementar los dos bloques pendientes	Registrar, modelar y probar los dos bloques que faltaban en el mod.	Allan	4	13/11/2026
F05	Implementar el cofre	Agregar el cofre al mod y comprobar su uso en el juego.	Allan	4	13/11/2026
F06	Documentar el código (Allan)	Documentar las clases y métodos de su parte del código.	Allan	3	13/11/2026
F07	Cierre de RF-21, RF-22 y RF-23	Terminar texturas de thorite ore, kyanita, cura y hongo, con las pruebas P07 a P09.	David	6	13/11/2026
F08	Demostración y opinión de usuarios	Preparar el mundo de demostración, mostrar el mod a los usuarios y grabar sus opiniones.	David y Fabio	3	13/11/2026
F09	Compilar el informe consolidado	Unir la documentación de todas las entregas en un solo PDF para Canvas.	David	4	12/11/2026

Calendario de la entrega final

Fecha	Fabio	Allan	David
12/11/2026	—	—	F09: informe consolidado.
13/11/2026	F01 a F03: errores, kyanita en la generación y documentación.	F04 a F06: dos bloques, cofre y documentación.	F07: cierre de RF-21, RF-22 y RF-23. F08 (con Fabio): demostración y usuarios.

5. Implementación

5.1 Arquitectura y clases

La arquitectura inicial, descrita en la Entrega 2, se representó en el diagrama UML de la figura 1. Desde entonces el sistema creció de forma incremental sobre esa base, por lo que el diagrama ilustra el núcleo original y la tabla 1 resume las clases que se fueron sumando.

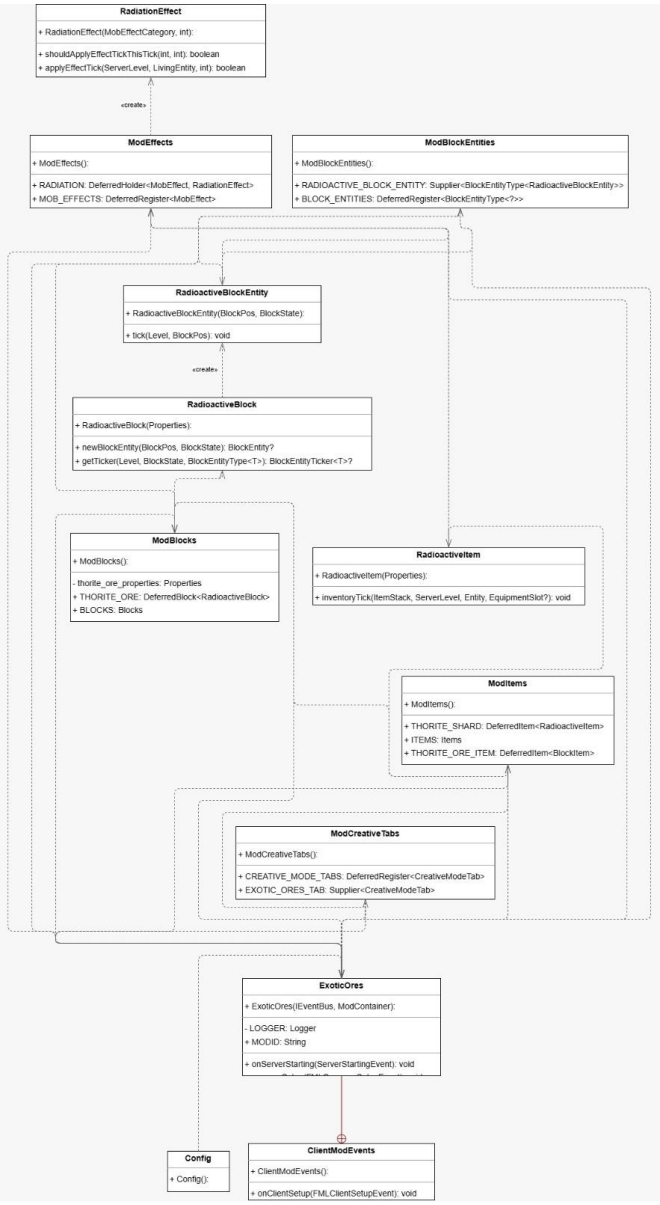


Figura 1. Diagrama UML de clases del núcleo del mod (Entrega 2).

Clase	Responsabilidad
ExoticOres	Clase principal. Inicializa el mod y coordina el registro de bloques, ítems, efectos y demás componentes.
ModBlocks, ModItems	Registran los bloques y los ítems del mod: Thorite, plomo, traje de radiación, armaduras y demás contenido.

Clase	Responsabilidad
ModEffects, RadiationEffect	Registran el efecto de radiación y determinan cuándo se aplica y qué hace sobre las entidades.
ModBlockEntities	Registra la entidad del bloque radiactivo y la relaciona con RadioactiveBlock.
RadioactiveBlock, RadioactiveBlockEntity, RadioactiveItem	Representan las fuentes de radiación: el bloque en el mundo, su lógica periódica (tick) y el objeto radiactivo en el inventario del jugador (inventoryTick).
Radioactive, RadiationBlockEmitter	Interfaces que separan la responsabilidad de ser fuente radiactiva de la de medir la exposición (Entrega 5).
RadiationExposure	Combina las contribuciones de todas las fuentes según la distancia y aplica el efecto una sola vez por tick. Incluye contributeHeal para la recuperación.
ModEffectEvents	Evento que aplica o bloquea el efecto de radiación, por ejemplo cuando las cuatro piezas equipadas pertenecen al tag radiation_protective.
ModArmorMaterial, ModTags	Definen el material de la armadura de Thorite y del traje de plomo, y los tags de protección y reparación.
ClearExceptProtectedConsumeEffect	Impide que la leche cure la radiación.
ModSounds, ModSoundProvider, GeigerTickClient	Registran el sonido del contador Geiger, generan sounds.json y reproducen los clics con mayor frecuencia cuanto mayor sea la intensidad.
RadiationIntensityPayload, ModNetworkEvents, RadiationScreenOverlay	Envían la intensidad del servidor al cliente y dibujan el efecto en pantalla (grano, viñeta e indicador en mSv/h).
ExoticOresConfig	Configuración del mod: activar o desactivar la radiación y multiplicar su rango.
ModConfiguredFeatures, ModPlacedFeatures, ModBiomeModifiers	Definen la forma, la colocación y los biomas de las vetas de Thorite en el End y de plomo en el Overworld.
ModModelProvider	Genera con datagen los modelos y estados de bloque y los conecta con las texturas.
ModCreativeTabs, ClientModEvents	Vista: pestaña creativa del mod y configuración del lado del cliente.

Tabla 1. Clases principales del sistema.

5.2 Evolución de la implementación

En la Entrega 2 se implementó el núcleo: el bloque radiactivo, su generación en el End, su daño por radiación y sus texturas. En la Entrega 3 se sumaron la armadura de Thorite, la mesa de refinado y el material refinado, y se afinó la lógica del efecto de radiación para que dependiera de la proximidad. La Entrega 4 convirtió ese prototipo en una experiencia completa: bloque de Thorite, radiación por intensidad centralizada en RadiationExposure, pantalla y sonido que avisan del peligro, traje de radiación de plomo, configuración y generación del plomo en el Overworld.

La Entrega 5 refactorizó el sistema de radiación con interfaces, agregó el veneno de las armas de plomo, el bloque de plomo y una nueva textura, y dejó en marcha la kyanita y la cura con hongo. Esa refactorización provocó un incidente de integración, descrito en la sección 8, que se resolvió revirtiendo los commits problemáticos.

Commits registrados en la Entrega 5

Requisito	Commit y fecha	Cambio registrado
RF-20	6d92645 (02/11/2026)	Allan: efecto Poison agregado a las armas de plomo.
RF-16	77759c4 (01/11/2026)	David: incorporación del bloque de plomo.
RF-21 y RF-08	792799a (01/11/2026)	David: nueva textura del bloque de plomo.

Requisito	Commit y fecha	Cambio registrado
RF-02, RF-05 y RF-13	ad78daf, 76c50a4, c753283 (26/10/2026)	Fabio: interfaces para ítems y bloques; adaptación de emisores y corrección de la emisión de contenedores.
RF-02 y RF-05	e69e09c, c92a263 (26/10/2026)	Fabio: adaptación de ModEffectEvents y corrección del amplificador de radiación.
RF-23	f9b450f (01/11/2026)	Fabio: contributeHeal en RadiationExposure y documentación (soporte de recuperación).
Integración	db4d94c (02/11), 3063b01 (03/11/2026)	Merges de los PR #18 y #19.

5.3 Trabajo de la entrega final

En la última etapa el equipo se dedicó a cerrar el sistema y a dejarlo documentado.

- **Fabio:** corrigió errores del mod, hizo que la generación utilice la kyanita (que amplía el requisito RF-22) y documentó parte del código.
- **Allan:** documentó la parte del código que le corresponde, implementó los dos bloques pendientes y agregó el cofre.
- **David:** continuó con las texturas, la kyanita, la cura y el hongo (RF-21, RF-22 y RF-23) y recopiló este informe consolidado.

5.4 Documentación del código

Cada integrante documentó las clases y métodos de su parte del código, de modo que se conozca la descripción de cada clase y su responsabilidad. La tabla 1 de este informe resume esas descripciones y el UML de la figura 1 muestra la estructura original.

5.5 Requisitos pendientes

Los siguientes requisitos no se implementaron y quedan para una futura iteración del mod.

Requisito	Qué falta
RF-06 Partículas y brillo	Diseñar y programar las partículas radiactivas de la mena y del bloque de Thorite.
RF-11 Mejora sobre netherita	Investigar e implementar la plantilla de herrería para mejorar la armadura de netherita a Thorite.
RF-17 Herramientas de plomo	Definir el material, registrar pico, hacha, pala y azada, crear sus tags y texturas y probarlas.
RF-18	Recuperar y aclarar su descripción original antes de planificarlo.
RF-19 Minibiomas de Thorite	Diseñar y registrar los minibiomas y concentrar en ellos la generación de la mena, que ahora se genera en todo el End.

6. Pruebas

Desde la Entrega 5 las pruebas se documentan con un procedimiento y un resultado esperado, y se vinculan a los requisitos y a las tareas. Probar y documentar son actividades distintas: no basta con ver que una mecánica funciona en una partida, sino que hay que dejar registrados la versión probada, los pasos y el resultado.

Prueba	Requisitos	Procedimiento	Resultado esperado
P01 Radiación	RF-02, RF-05, RF-13	Acercar y alejar a un jugador sin protección de una mena o bloque radiactivo; repetir con más de una fuente.	Exposición y daño acordes a la intensidad; sin aumentos indebidos del amplificador ni aplicación duplicada.
P02 Material transportado	RF-04, RF-13	Comparar inventario sin mineral, con material radiactivo y con el material retirado; revisar contenedores.	Las fuentes aportan exposición según las reglas; al retirarlas no quedan contribuciones erróneas.
P03 HUD, sonido y configuración	RF-07, RF-09, RF-14	Variar la exposición; observar HUD y escuchar el Geiger; desactivar la radiación y cambiar su rango.	Pantalla y sonido reflejan la intensidad; la desactivación y el rango se respetan.
P04 Generación de minerales	RF-01, RF-16	Generar un mundo o chunks nuevos; buscar Thorite en el End y plomo en el Overworld; minar y procesar.	Ambas menas aparecen en su dimensión; sus drops y procesamiento permiten continuar la progresión.
P05 Equipamiento	RF-11, RF-15	Comparar exposición sin armadura, con traje de plomo completo, incompleto y armadura de Thorite.	La protección corresponde a las piezas requeridas; un conjunto incompleto no da protección total.
P06 Armas de plomo	RF-20	Golpear a un objetivo susceptible con cada arma; comparar con otra arma, con portar sin atacar y con un objetivo inmune.	Poison se aplica solo en los casos previstos, con duración e intensidad configuradas; se conserva el daño.
P07 Texturas	RF-21, RF-08	Revisar el bloque de plomo y la mena de Thorite en inventario y mundo, todas sus caras y modelos.	Texturas actualizadas, sin caras incorrectas ni recursos faltantes.
P08 Bloques	RF-16, RF-22	Obtener, colocar y minar los bloques de plomo y kyanita; revisar su modelo.	Los recursos cargan sin texturas faltantes; los bloques se colocan y recuperan según su diseño.
P09 Cura y hongo	RF-23	Aplicar la cura mediante el hongo; comparar exposición antes y después, fuera y dentro del alcance de una fuente.	La recuperación coincide con el efecto diseñado y se integra con RadiationExposure.

7. Control de versiones

El código se aloja en GitHub: **https://github.com/astxrfr/ExoticOres.git**. Cada integrante cuenta con su propio usuario (Fabio: astxrfr, David: eldeivid68, Allan: B-All19) y los tres trabajan sobre el mismo código. La meta del equipo es registrar más de dos commits por semana por integrante durante todo el desarrollo.

A partir de la Entrega 5 cada commit se vincula con su requisito, su tarea y su prueba, lo que permite revisar el origen de cada cambio y revertirlo con seguridad cuando una integración falla.

8. Reflexión de las entregas

A lo largo del semestre el equipo fue encontrando dificultades distintas en cada etapa. En las primeras, el obstáculo principal fue entender el proyecto y la API; con el tiempo apareció el reto de mantener el trabajo de tres personas integrado y comprobado.

8.1 Dificultades y solución por entrega

Entrega	Dificultad	Cómo se resolvió
Entrega 2	La clase RadioactiveBlock y su funcionalidad fueron difíciles de comprender; la integración de texturas resultó estresante y la API de NeoForge dio problemas durante las pruebas.	Se trabajó en equipo: mientras se elaboraba el UML, Fabio fue explicando la estructura de las clases, y las pruebas y la depuración permitieron dejar funcionando el bloque, las texturas y la generación.
Entrega 3	Al vincular el mineral al bioma del End, el juego no reconocía el bloque.	Se ajustó la vinculación hasta que el mineral se generó de forma natural en el End; el efecto de radiación se afinó y se probó en el juego.
Entrega 4	El proyecto era pesado por la cantidad de clases, registros y archivos; el código era difícil de entender al inicio para parte del equipo; el entorno funcionaba en la computadora de un integrante y en las otras no.	Se dividió el trabajo en tareas pequeñas por integrante; quienes dominaban más el código explicaron clases como RadioactiveBlockEntity y RadiationEffect, con apoyo del UML; se unificaron las versiones de Java, Minecraft y NeoForge y se compartió la configuración por GitHub.
Entrega 4 (técnicas)	La radiación se aplicaba doble por tick; el payload de red no llegaba al cliente; el datagen mostraba errores; se olvidó generar sounds.json; hubo un conflicto menor en GitHub; el plomo salía a una altura poco útil.	La radiación se centralizó en RadiationExposure, que suma las contribuciones y aplica el efecto una sola vez; se corrigió el envío hasta que la intensidad llegó al cliente; se resolvió el error del datagen; se agregó el sounds.json faltante; el conflicto se resolvió al integrar y se ajustó la altura de generación del plomo.
Entrega 5	Funciones ya implementadas, como la radiación, dejaron de funcionar después de integrar commits.	Se revirtieron los commits que causaron el problema y se recuperó una versión funcional, sin cambiar la mecánica; se revisó el trabajo en la versión compartida.

8.2 Lecciones aprendidas

- Dividir cada requisito en tareas pequeñas con un responsable claro permite que los tres avancemos en paralelo sin estorbarnos.
- Usar el sistema de persistencia nativo de Minecraft evitó una sobreingeniería, ya que todo el estado del mod vive en el mundo del juego.
- Centralizar la radiación en RadiationExposure permite que el HUD, el sonido y la protección reaccionen a la misma intensidad sin duplicar lógica.
- Priorizar lo que completa la experiencia principal (radiación, protección y plomo) antes de agregar contenido adicional ayuda a entregar un prototipo funcional.
- Probar cada tarea antes de integrarla ahorra tiempo, porque los errores se encuentran cuando todavía son fáciles de ubicar.
- Mantener cambios pequeños y relacionados con una tarea facilita identificar qué provocó un error y qué se perdería al revertirlo. Por eso cada commit debe incluir el requisito y la tarea.

- Probar y documentar son actividades diferentes: hay que guardar la versión probada, los pasos y el resultado para poder comparar antes y después de una refactorización.
- Cambiar un archivo de textura no basta: hay que comprobar que el juego lo muestre correctamente, y la planificación debe reflejar el estado real del trabajo.

9. Opinión de los usuarios

La participación de los usuarios comenzó desde la Entrega 1, cuando los tres prototipos del material se mostraron a posibles usuarios (sección 1.6). Pidieron un diseño más extravagante pero comprensible y colores vivos tipo neón, y preguntaron por el propósito del material; la preocupación por una textura coherente con la estética del End quedó recogida en el requisito RF-08 de la Entrega 2.

El mod se mostró a usuarios potenciales en las entregas 4 y 5, y las sesiones quedaron grabadas en video. En la cuarta entrega se grabaron tres videos y en la quinta, cuatro. En ellos los usuarios probaron el mod y dieron su opinión.

En general la opinión es positiva. A los usuarios les agradó el contenido agregado al End, consideraron interesantes las mecánicas del nuevo mineral y creativas las nuevas armas. Entre sus propuestas de mejora mencionaron agregar un mob nuevo y habilidades especiales a las armas. Estas propuestas no entraron en el alcance de esta entrega y quedan registradas como ideas para trabajo futuro. Como conclusión, la recepción del mod fue buena.

Videos de la interacción con los usuarios

Entrega 4:

- https://youtu.be/DZp7dRB9Tnw
- https://youtu.be/5_TOfmm5jyw
- https://youtu.be/yp6HkGrKLl8

Entrega 5:

- https://youtu.be/2HmdecXvMWU
- https://youtu.be/TZS8Hg5LJiI
- https://youtu.be/eMMQCCRThBc
- https://youtu.be/xIy9WMgMEMA

Entrega Final

- https://youtu.be/VxD8_CE8uBQ?si=Ed92m1givNgbxDG6
- https://youtu.be/8Z0cFU_9mxI

10. Gestión del tiempo

Cada integrante registró en el Formulario 1 la fecha, la hora de inicio y fin, las interrupciones y el tiempo real dedicado a cada tarea, calculado como (Fin − Inicio) − Interrupción. La siguiente tabla resume los minutos trabajados por entrega.

Entrega	Fabio (min)	Allan (min)	David (min)	Observación
Entrega 1	75	340	570	Según el formulario original; el tiempo de Fabio en los mapas de empatía no se registró y varias filas no coinciden con la fórmula (Fin − Inicio) − Interrupción.
Entrega 2	—	—	—	Registro conjunto del equipo: 1740 min.
Entrega 3	560	255	920	
Entrega 4	600	500	620	
Entrega 5	200	255	480	Total de David recalculado con la fórmula del formulario (el formulario original reportaba 660).
Entrega final	Por registrar	Por registrar	Por registrar	Ver los formularios siguientes.

Formulario 1 – Fabio Roberto Calvo Lemus (carné 262256) · Entrega final

Fecha	Inicio	Fin	Interrupción (min)	Delta tiempo (min)	Tarea	Comentarios
12/11/2026	2:00 pm	2:40 pm	0	40	Corregir errores	Hubieron pocos y la mayoria fueron de logica
13/11/2026	4:00 pm	6:00 pm	20	100	Generación con kyanita	Fue interesante la forma de hacer que se genere y que haya una probabilidad que cambie
13/11/2026	6:00 pm	7:00 pm	0	60	Documentar el código	Me gusta ver el fruto de nuestro esfuerzo como equipo

Formulario 1 – Allan Enrique Ronneseth Quemé (carné 26493) · Entrega final

Fecha	Inicio	Fin	Interrupción (min)	Delta tiempo (min)	Tarea	Comentarios
12/11/26	8pm	8:20pm	0min	20min	Documentar el código	Facilito
12/11/26	9pm	11:50pm	30 min	140 min	Implementar el cofre	Si es bastante para agregar un solo cofre

Formulario 1 – David Adriel Aguirre Leonzo (carné 261134) · Entrega final

Fecha	Inicio	Fin	Interrupción (min)	Delta tiempo (min)	Tarea	Comentarios
12/11/26	2:00	11:00	0	480	Cierre de RF-21, RF-22 y RF-23	Fue interesante la forma en que se implemente el tiempo ya que es por medio de thicks y no por tiempo
12/11/26	9:00	9:30	0	30	Compilar el informe Final	

11. Material de entrega

- **Repositorio de GitHub:** https://github.com/astxrfr/ExoticOres.git
- **Videos de la interacción con los usuarios:** enlaces en la sección 9.
- https://youtu.be/VxD8_CE8uBQ?si=Ed92m1givNgbxDG6
- https://youtu.be/8Z0cFU_9mxI